

Kubernetes Documentation: Basic to Expert Level

1. What is Kubernetes?

Kubernetes, also called K8s, is a container orchestration platform. It helps you run applications in containers and manage them automatically.

In simple words:

Docker creates containers. Kubernetes manages many containers across many servers.

Kubernetes helps with:

- Deployment
- Scaling
- Load balancing
- Self-healing
- Rolling updates
- Rollbacks
- Service discovery
- Configuration management
- Secrets management
- High availability

2. Why Kubernetes is Needed

Without Kubernetes:

- You manually start containers
- You manually restart failed containers
- You manually manage traffic
- You manually scale applications
- You manually update applications

With Kubernetes:

- Application crashes → Kubernetes restarts it
- More users come → Kubernetes can scale pods
- New version deployed → Kubernetes performs rolling update
- Old version needed → Kubernetes can rollback
- One server fails → Kubernetes schedules workload elsewhere

3. Real-World Example

Imagine an e-commerce website.

Components:

- Frontend service
- Backend API service
- Payment service
- Order service
- Database
- Redis cache

During normal traffic, backend may run 3 pods.

During sale traffic, backend may need 20 pods.

Kubernetes handles this by scaling pods horizontally.

4. Core Kubernetes Architecture

A Kubernetes cluster has two major parts:

1. Control Plane
2. Worker Nodes

4.1 Control Plane

Control Plane is the brain of Kubernetes.

Main components:

API Server

The entry point of Kubernetes.

When you run:

```
kubectl get pods
```

Your request goes to API Server.

Scheduler

Decides on which worker node a pod should run.

Example:

- Node 1 has CPU free
- Node 2 is full
- Scheduler places pod on Node 1

Controller Manager

Keeps desired state and actual state the same.

Example:

You want 5 pods. One pod dies. Controller Manager tells Kubernetes to create one more pod.

etcd

Kubernetes database.

It stores cluster state:

- Nodes
- Pods
- Services
- Secrets
- ConfigMaps
- Deployments

5. Worker Node Components

Worker nodes run your application.

Kubelet

Agent running on each node. It talks to the Control Plane and ensures pods are running.

Container Runtime

Runs containers.

Examples:

- containerd
- CRI-O
- Docker runtime older usage

Kube Proxy

Handles networking and service routing inside the cluster.

6. Important Kubernetes Objects

6.1 Pod

Pod is the smallest deployable unit in Kubernetes.

A pod contains one or more containers.

Usually:

```
1 Pod = 1 Application Container
```

Example:

```
kubectl run nginx-pod --image=nginx
```

Check pod:

```
kubectl get pods
```

6.2 Deployment

Deployment manages pods.

It provides:

- Replica management
- Rolling update
- Rollback
- Self-healing

Example:

```
kubectl create deployment nginx-app --image=nginx
```

Check deployment:

```
kubectl get deployments
```

6.3 ReplicaSet

ReplicaSet ensures required number of pod replicas are running.

Usually, you do not create ReplicaSet directly. Deployment creates and manages ReplicaSet.

Example:

```
Deployment → ReplicaSet → Pods
```

6.4 Service

Pod IPs are temporary. If pod restarts, IP changes.

Service gives a stable network endpoint.

Types of Services:

ClusterIP

Default service type. Used for internal communication inside the cluster.

Example:

```
kubectl expose deployment nginx-app --type=ClusterIP --port=80
```

NodePort

Exposes service on worker node port.

Useful for local labs.

```
kubectl expose deployment nginx-app --type=NodePort --port=80
```

LoadBalancer

Used in cloud environments like AKS, EKS, GKE.

It creates external cloud load balancer.

```
kubectl expose deployment nginx-app --type=LoadBalancer --port=80
```

6.5 ConfigMap

Used to store non-sensitive configuration.

Example:

```
kubectl create configmap app-config --from-literal=APP_ENV=dev
```

6.6 Secret

Used to store sensitive data like passwords, tokens, API keys.

Example:

```
kubectl create secret generic db-secret --from-literal=password=MyPassword123
```

6.7 Namespace

Namespace is used to logically separate resources.

Example:

```
kubectl create namespace dev  
kubectl get pods -n dev
```

Real-world example:

- dev namespace
- test namespace
- staging namespace
- production namespace

6.8 Ingress

Ingress exposes HTTP/HTTPS applications using rules.

Example:

```
shop.example.com    → frontend-service  
api.example.com     → backend-service  
payment.example.com → payment-service
```

Ingress needs an Ingress Controller like:

- NGINX Ingress Controller
- Traefik
- Azure Application Gateway Ingress Controller
- AWS ALB Ingress Controller

7. Local Lab Setup on Windows

Recommended tools:

- Docker Desktop
- kind
- kubectl
- PowerShell

7.1 Check Docker

```
docker --version
```

Check Docker is running:

```
docker ps
```

7.2 Install kind

Using winget:

```
winget install Kubernetes.kind
```

Important: after installation, close PowerShell and open it again.

Check:

```
kind version
```

If kind is not recognized, refresh PATH:

```
$env:Path = [System.Environment]::GetEnvironmentVariable("Path","Machine") +  
";" + [System.Environment]::GetEnvironmentVariable("Path","User")  
kind version
```

7.3 Install kubectl

```
winget install Kubernetes.kubectl
```

Check:

```
kubectl version --client
```

8. Practical Lab 1: Create Local Kubernetes Cluster

```
kind create cluster --name satish-lab
```

Check cluster:

```
kubectl cluster-info  
kubectl get nodes
```

Expected:

```
satish-lab-control-plane   Ready
```

9. Practical Lab 2: Deploy Nginx Application

```
kubectl create deployment nginx-app --image=nginx
```

Check:

```
kubectl get deployments  
kubectl get pods
```

10. Practical Lab 3: Expose Application

```
kubectl expose deployment nginx-app --type=NodePort --port=80
```

Check service:

```
kubectl get svc
```

Use port-forward:

```
kubectl port-forward service/nginx-app 8080:80
```

Open browser:

```
http://localhost:8080
```

You should see nginx welcome page.

11. Practical Lab 4: Scale Application

Scale from 1 pod to 5 pods:


```
kubectl scale deployment nginx-app --replicas=5
```

Check:

```
kubectl get pods
```

Meaning:

```
1 application copy became 5 application copies
```

This is horizontal scaling.

12. Practical Lab 5: Self-Healing Test

Get pods:

```
kubectl get pods
```

Delete one pod:

```
kubectl delete pod <pod-name>
```

Check again:

```
kubectl get pods
```

Kubernetes will create a new pod automatically.

This is self-healing.

13. Practical Lab 6: Rolling Update

Update nginx image:

```
kubectl set image deployment/nginx-app nginx=nginx:1.27
```

Check rollout:

```
kubectl rollout status deployment/nginx-app
```

Check history:

```
kubectl rollout history deployment/nginx-app
```

Rollback:

```
kubectl rollout undo deployment/nginx-app
```

14. Practical Lab 7: YAML Deployment

Create file:

```
notepad nginx-deployment.yaml
```

Paste:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-yaml-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-yaml-app
  template:
    metadata:
      labels:
        app: nginx-yaml-app
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

Apply:

```
kubectl apply -f nginx-deployment.yaml
```

Check:

```
kubectl get deployments
kubectl get pods
```

15. Practical Lab 8: YAML Service

Create file:

```
notepad nginx-service.yaml
```

Paste:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-yaml-service
spec:
  type: NodePort
  selector:
    app: nginx-yaml-app
  ports:
    - port: 80
      targetPort: 80
```

Apply:

```
kubectl apply -f nginx-service.yaml
```

Check:

```
kubectl get svc
```

Port forward:

```
kubectl port-forward service/nginx-yaml-service 8081:80
```

Open:

```
http://localhost:8081
```

16. Practical Lab 9: ConfigMap

Create ConfigMap:

```
kubectl create configmap app-config --from-literal=APP_ENV=local --from-literal=APP_OWNER=Satish
```

Check:

```
kubectl get configmap  
kubectl describe configmap app-config
```

17. Practical Lab 10: Secret

Create secret:

```
kubectl create secret generic db-secret --from-literal=DB_PASSWORD=Password123
```

Check:

```
kubectl get secret  
kubectl describe secret db-secret
```

18. Practical Lab 11: Namespace

Create namespace:

```
kubectl create namespace dev
```

Deploy app inside namespace:

```
kubectl create deployment dev-nginx --image=nginx -n dev
```

Check:

```
kubectl get pods -n dev
```

19. Practical Lab 12: Resource Requests and Limits

Create file:

```
notepad resource-demo.yaml
```

Paste:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: resource-demo
  template:
    metadata:
      labels:
        app: resource-demo
    spec:
      containers:
      - name: nginx
        image: nginx
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "256Mi"
```

Apply:

```
kubectl apply -f resource-demo.yaml
```

Meaning:

- Request = minimum guaranteed resource
- Limit = maximum allowed resource

20. Practical Lab 13: Liveness and Readiness Probe

Create file:

```
notepad probe-demo.yaml
```

Paste:

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: probe-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: probe-demo
  template:
    metadata:
      labels:
        app: probe-demo
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 15
          periodSeconds: 20
```

Apply:

```
kubectl apply -f probe-demo.yaml
```

Meaning:

- Readiness probe: decides whether pod can receive traffic
- Liveness probe: decides whether pod should be restarted

21. Practical Lab 14: Basic Troubleshooting

Useful commands:

```
kubectl get pods
kubectl get pods -o wide
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- /bin/sh
```

```
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl describe deployment <deployment-name>
kubectl describe svc <service-name>
```

Common errors:

ImagePullBackOff

Meaning: Kubernetes cannot pull image.

Check:

```
kubectl describe pod <pod-name>
```

Possible reasons:

- Wrong image name
- Private registry authentication issue
- Internet issue

CrashLoopBackOff

Meaning: container starts and crashes repeatedly.

Check logs:

```
kubectl logs <pod-name>
```

Pending Pod

Meaning: pod is not scheduled.

Possible reasons:

- Not enough CPU/memory
- Node issue
- Taints/tolerations issue

22. Helm Basics

Helm is a package manager for Kubernetes.

Example:

```
APT/YUM for Linux
Helm for Kubernetes
```

Install chart:

```
helm install my-nginx bitnami/nginx
```

Check releases:

```
helm list
```

Uninstall:

```
helm uninstall my-nginx
```

Real-world use:

- Install NGINX ingress
- Install Prometheus
- Install Grafana
- Install cert-manager

23. Kubernetes Networking Basics

Important networking concepts:

- Pod-to-pod communication
- Service-to-pod communication
- Ingress-to-service routing
- DNS-based service discovery

Example:

```
User → Ingress → Service → Pod
```

Internal app communication:

```
frontend → backend-service → backend pods
```

24. Kubernetes Storage Basics

Containers are temporary. If pod dies, data inside container can be lost.

For persistent data, Kubernetes uses:

- Volume
- PersistentVolume

- PersistentVolumeClaim
- StorageClass

Real-world:

- Database needs persistent storage
- Logs may need persistent storage
- File uploads need persistent storage

25. Kubernetes Security Basics

Important topics:

- RBAC
- ServiceAccount
- NetworkPolicy
- Secrets
- Pod Security Standards
- Image scanning
- Runtime security
- Least privilege access

Real-world best practices:

- Do not run containers as root
- Use resource limits
- Use private registry
- Scan images
- Restrict namespaces
- Use RBAC roles properly
- Protect kubeconfig

26. Monitoring and Logging

Common tools:

- Prometheus
- Grafana
- Loki
- EFK stack
- OpenTelemetry

What to monitor:

- Node CPU
- Node memory
- Pod CPU
- Pod memory
- Restart count
- Failed pods
- API server health

- Application latency

27. Production Architecture

A production Kubernetes setup usually includes:

- Multi-node cluster
- Multiple availability zones
- Ingress controller
- TLS certificate management
- Monitoring stack
- Logging stack
- CI/CD pipeline
- RBAC
- Backup strategy
- Autoscaling
- Resource quotas
- Network policies

28. AKS/EKS/GKE Architect View

Managed Kubernetes services:

- AKS: Azure Kubernetes Service
- EKS: Elastic Kubernetes Service
- GKE: Google Kubernetes Engine

Cloud provider manages control plane.

You manage:

- Worker nodes
- Applications
- Networking
- Security
- Monitoring
- Scaling
- Cost optimization

29. Kubernetes with Terraform

Terraform can create Kubernetes infrastructure.

Example responsibilities:

- Create resource group
- Create VNet/subnet
- Create AKS cluster
- Create node pools
- Configure RBAC

- Configure monitoring

Kubernetes YAML/Helm then deploys application workloads.

Simple division:

```
Terraform → Infrastructure  
Helm/YAML → Applications
```

30. Kubernetes with CI/CD

Typical pipeline:

1. Developer commits code
2. Pipeline builds Docker image
3. Image pushed to container registry
4. Kubernetes manifest updated
5. Deployment applied to cluster
6. Rolling update starts
7. Monitoring validates health

Tools:

- Azure DevOps
- GitHub Actions
- GitLab CI
- Jenkins
- Argo CD
- Flux CD

31. Beginner to Expert Learning Path

Level 1: Beginner

Focus:

- Pod
- Deployment
- Service
- Namespace
- kubectl commands

Practice:

- Deploy nginx
- Expose nginx
- Scale nginx
- Delete pod and observe self-healing

Level 2: Intermediate

Focus:

- YAML manifests
- ConfigMap
- Secret
- Resource requests/limits
- Probes
- Rolling update
- Rollback

Practice:

- Write deployment YAML
- Add service YAML
- Add probes
- Perform rollout and rollback

Level 3: Advanced

Focus:

- Ingress
- Helm
- Storage
- RBAC
- NetworkPolicy
- HPA
- Monitoring

Practice:

- Install ingress controller
- Deploy app with Helm
- Configure HPA
- Configure Prometheus/Grafana

Level 4: Expert / Architect

Focus:

- AKS/EKS/GKE design
- Cluster networking
- Security architecture
- GitOps
- Multi-environment strategy
- Disaster recovery
- Cost optimization
- Platform engineering

Practice:

- Design production AKS architecture
- Use Terraform for cluster provisioning
- Use Helm/Argo CD for app deployment
- Implement monitoring and alerting
- Define RBAC and namespace strategy

32. Interview Explanation

What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications.

What is Pod?

Pod is the smallest deployable unit in Kubernetes. It runs one or more containers.

What is Deployment?

Deployment manages pods and provides scaling, rolling updates, rollback, and self-healing.

What is Service?

Service provides a stable network endpoint to access pods.

What is Ingress?

Ingress manages HTTP/HTTPS external access to services using host/path rules.

What is HPA?

Horizontal Pod Autoscaler automatically increases or decreases pod count based on CPU, memory, or custom metrics.

33. Real-World Architect Statement

I design Kubernetes platforms by separating infrastructure and application layers. Terraform provisions the cloud infrastructure and managed Kubernetes cluster, while Helm or GitOps tools deploy application workloads. I focus on scalability, high availability, RBAC, network security, monitoring, logging, cost optimization, and disaster recovery.

34. Cleanup Commands

Delete apps:

```
kubectl delete deployment nginx-app  
kubectl delete service nginx-app
```

Delete namespace:

```
kubectl delete namespace dev
```

Delete kind cluster:

```
kind delete cluster --name satish-lab
```

35. Daily Practice Plan

Day 1

- Install Docker, kind, kubectl
- Create local cluster
- Deploy nginx
- Expose service
- Scale pods

Day 2

- Learn YAML
- Deployment YAML
- Service YAML
- ConfigMap
- Secret

Day 3

- Probes
- Resource limits
- Rolling update
- Rollback

Day 4

- Namespace
- RBAC basics
- Troubleshooting

Day 5

- Ingress
- Helm
- Monitoring basics

Day 6

- HPA
- Storage
- NetworkPolicy

Day 7

- AKS/EKS/GKE architecture
- Terraform integration
- CI/CD deployment flow

36. Most Important Commands Cheat Sheet

```
kubectl get nodes
kubectl get pods
kubectl get pods -o wide
kubectl get deployments
kubectl get svc
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- /bin/sh
kubectl apply -f file.yaml
kubectl delete -f file.yaml
kubectl scale deployment <name> --replicas=5
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl get events --sort-by=.metadata.creationTimestamp
```

37. Final Summary

Kubernetes is not just a tool. It is a platform for running modern applications reliably.

Basic level means you can deploy and expose apps.

Intermediate level means you can write YAML, manage configs, secrets, probes, and rollouts.

Advanced level means you can handle ingress, Helm, storage, autoscaling, monitoring, and troubleshooting.

Expert level means you can design secure, scalable, production-grade Kubernetes platforms on cloud using Terraform, CI/CD, GitOps, monitoring, and governance.